

100 Core Java Programming Assignment Questions

Beginner Level (Basics, Control Structures)

1. Write a Java program to print "Hello, World!".
2. Write a program to print the sum of two numbers.
3. Write a program to swap two numbers using a temporary variable.
4. Write a program to check if a number is even or odd.
5. Write a program to check whether a number is positive, negative, or zero.
6. Write a program to find the largest of three numbers.
7. Write a program to calculate the factorial of a number.
8. Write a program to generate the Fibonacci series up to `n` terms.
9. Write a program to reverse a number.
10. Write a program to check if a number is a palindrome.
11. Write a program to find the sum of digits in a number.
12. Write a program to check if a number is prime.
13. Write a program to print all prime numbers between 1 and 100.
14. Write a program to check if a year is a leap year.
15. Write a program to find the GCD and LCM of two numbers.
16. Write a program to print the multiplication table of any number.
17. Write a program to calculate the power of a number using loops.
18. Write a program to calculate the area and perimeter of a rectangle.
19. Write a program to implement a simple calculator using switch-case.
20. Write a program to display the ASCII value of a character.

Intermediate Level (Arrays, Strings, and Methods)

1. Write a program to declare and initialize an array.
2. Find the largest element in an array.
3. Find the smallest element in an array.

4. Calculate the sum and average of the elements in an array.
5. Program to reverse an array.
6. Sort an array in ascending order.
7. Perform linear search in an array.
8. Perform binary search in a sorted array.
9. Remove duplicate elements from an array.
10. Copy all elements from one array to another.
11. Program to check if two arrays are equal.
12. Write a program to find the frequency of each element in an array.
13. Split a string into words.
14. Reverse a string using loops.
15. Check whether a string is a palindrome.
16. Count the number of vowels and consonants in a string.
17. Count the frequency of characters in a string.
18. Remove all whitespaces from a string.
19. Write a program to convert a string to uppercase and lowercase.
20. Write a method to find the factorial of a number using recursion.

Object-Oriented Programming (Classes, Objects, Inheritance, Polymorphism)

1. Create a class to print student details using a constructor.
2. Demonstrate method overloading.
3. Demonstrate method overriding.
4. Implement single inheritance.
5. Implement multilevel inheritance.
6. Implement hierarchical inheritance.
7. Demonstrate the use of `this` keyword.
8. Use `super` to call the parent class constructor.
9. Create an abstract class and implement it using a subclass.

10. Implement an interface in a class.
11. Demonstrate multiple inheritance through interfaces.
12. Write a program to demonstrate static variables and methods.
13. Create a class with an inner class and access its methods.
14. Use encapsulation to protect data.
15. Create a program to demonstrate the use of constructors (default and parameterized).
16. Create a class to implement constructor overloading.
17. Write a program to demonstrate the use of final classes and methods.
18. Demonstrate the use of polymorphism in Java.
19. Create a program illustrating `instanceof` operator.
20. Implement a simple banking application using classes and objects.

Collections Framework

1. Demonstrate the use of `ArrayList`.
2. Demonstrate the use of `LinkedList`.
3. Demonstrate the use of `HashSet` and `TreeSet`.
4. Demonstrate the use of `HashMap` and `TreeMap`.
5. Sort elements in an `ArrayList`.
6. Iterate through a `Map` using `for-each` loop and `Iterator`.
7. Remove duplicates from a `List` using a `Set`.
8. Store and retrieve elements from a `LinkedHashMap`.
9. Compare `ArrayList` and `LinkedList`.
10. Create a program to sort a `Map` by its keys.
11. Convert a `List` to an array and vice versa.
12. Find the frequency of elements in a `List`.
13. Demonstrate the use of stack using `Stack` class.
14. Implement a queue using `LinkedList`.
15. Write a program to iterate over a `HashSet`.

Exception Handling

1. Demonstrate the use of try-catch blocks.
2. Handle multiple exceptions in a single program.
3. Demonstrate the use of `finally` block.
4. Create a custom exception and handle it.
5. Use `throw` and `throws` keywords in a program.
6. Handle `ArithmeticException` while dividing by zero.
7. Handle `NullPointerException`.
8. Handle `ArrayIndexOutOfBoundsException`.
9. Create a program to illustrate nested try-catch blocks.
10. Demonstrate checked vs unchecked exceptions.

File Handling

1. Write a program to create and write text to a file.
2. Read data from a file using `FileReader`.
3. Append text to an existing file.
4. Copy content from one file to another.
5. Count the number of words in a file.
6. Read a file line by line using `BufferedReader`.
7. Demonstrate serialization and deserialization.
8. Program to list all files and directories.
9. Delete a file from the system.
10. Program to implement random access file operations.

Multithreading and Synchronization

1. Create a thread by extending the `Thread` class.
2. Create a thread by implementing the `Runnable` interface.
3. Demonstrate thread synchronization using the `synchronized` keyword.
4. Create a program to implement inter-thread communication using `wait()` and `notify()`.

5. Create a program with multiple threads printing numbers concurrently.